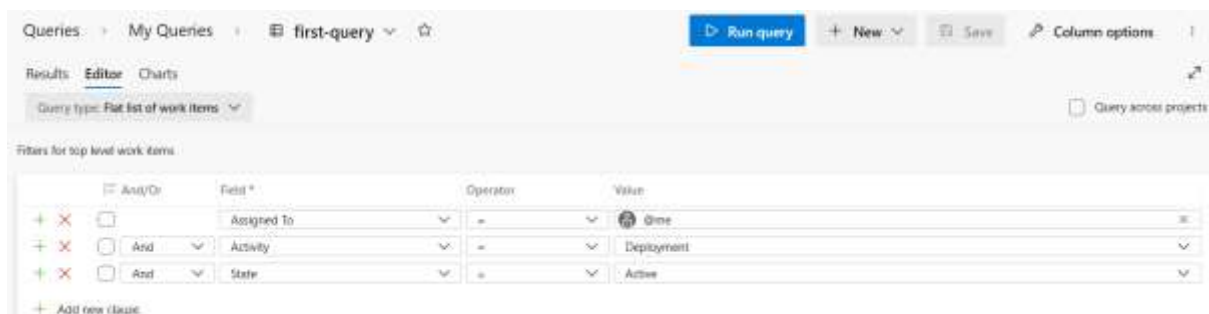


Name: Kriti
College: Lovely Professional University
Week7 Assignment
Email: kritijulia@gmail.com
Student_Id: CT_CSI_DV_5113

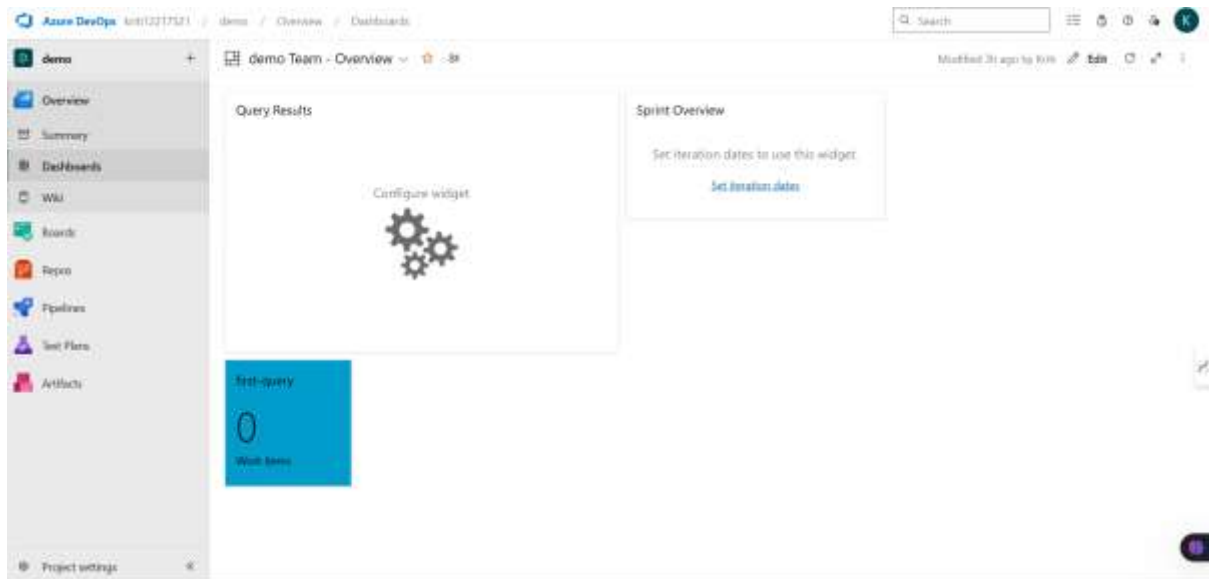
1. Configure Dashboard and Queries for Work Items

Steps:

- Go to your Azure DevOps project.
- Navigate to **Boards > Queries**. Create a new query (e.g., Assigned To = @Me AND State = Active).
- Save the query and pin it to the dashboard.
- Go to **Dashboards**, click on **New Dashboard** or edit an existing one.
- Add widgets like "Query Results", "Sprint Overview", or "Work Items" and link them to your saved queries.



Query creation screen.



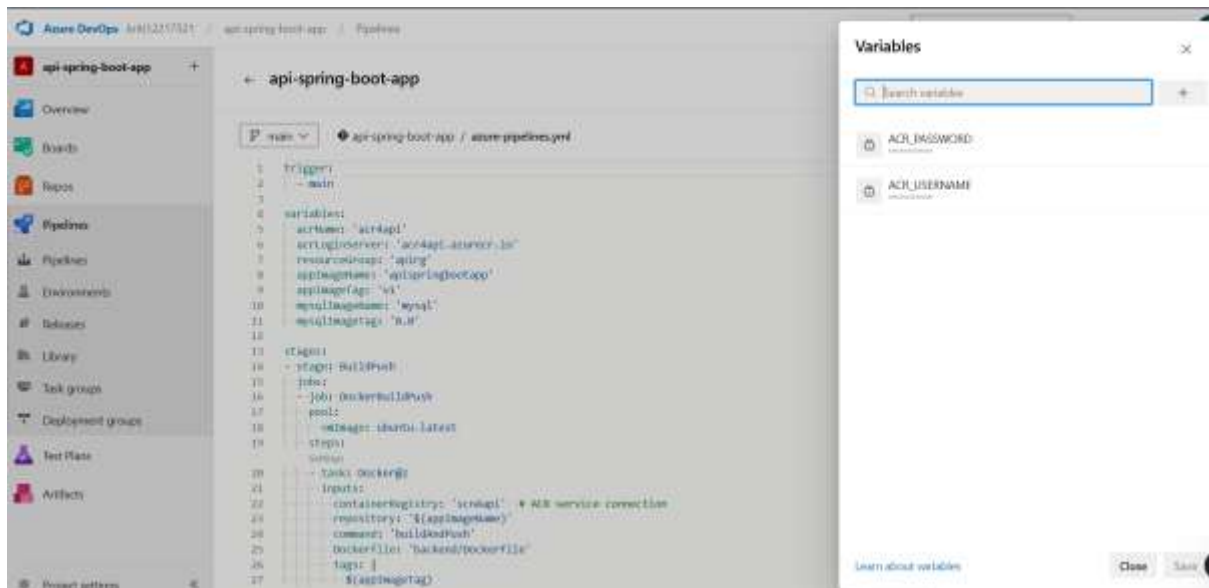
Dashboard with widgets showing work items.

Here, I configured a custom query to track active work items assigned to me and visualized them using a dashboard. This helps stakeholders get a quick overview of sprint progress.

2. Use Pipeline Variables While Configuring Pipelines

Steps:

- Navigate to **Pipelines > Pipelines**, create/edit a YAML pipeline.
- Define variables in YAML using variables: block or use the **UI > Variables** tab.
- Use variables with `$(varName)` syntax in pipeline steps.



Variable defined in YAML or GUI and Usage of variable in pipeline script.

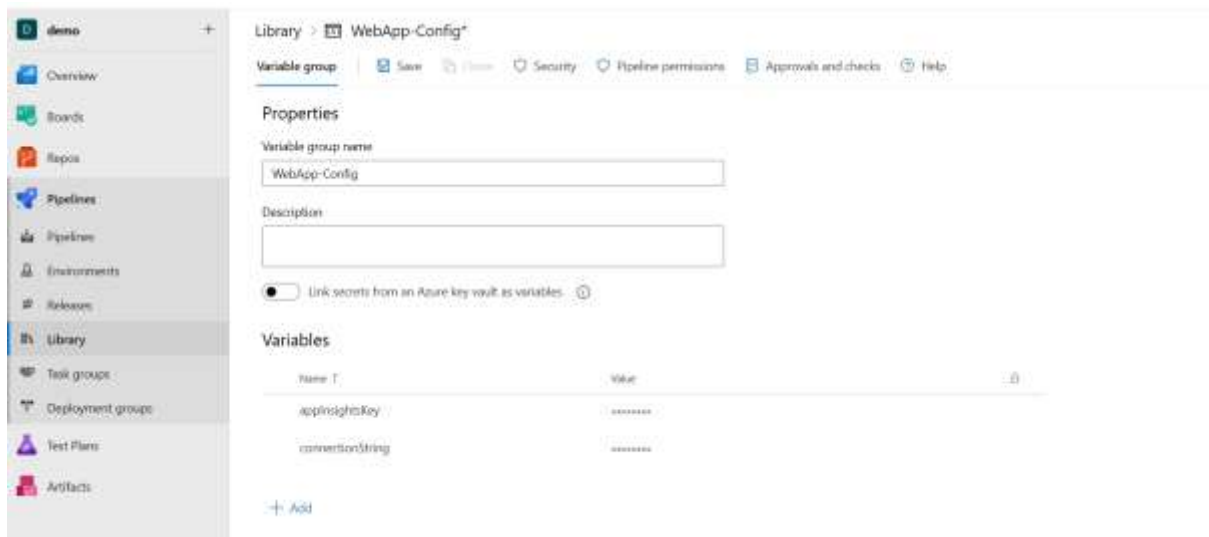
Using variables makes my pipeline reusable and maintainable. For example, I used a variable to define the build configuration, which can be changed centrally without editing every step.

3. Use Variable and Task Groups in Pipelines and Set Scopes

Steps:

- Go to **Pipelines > Library**.
- Create a **Variable Group**, add variables like connectionString, envName, etc.
- Link variable group to pipeline and set scope at stage-level if needed.
- Create a **Task Group** from a pipeline task sequence and reuse it.

Variable group creation.



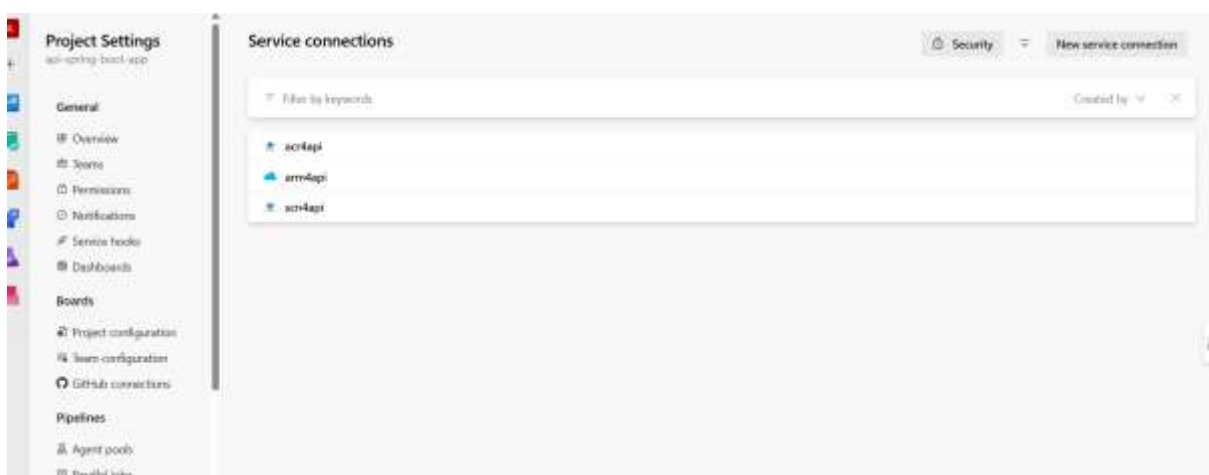
Linking variable group in a pipeline and Task group showing reusable steps.

I modularized my pipeline using variable groups (shared across pipelines) and task groups (shared task logic), improving reusability and maintainability. Scoped variables ensure stage-specific configurations.

4. Create a Service Connection

Steps:

- Go to **Project Settings > Service connections**.
- Click **New service connection** > Choose **Azure Resource Manager** > Select authentication method (e.g., Service Principal).
- Grant access permission to pipelines.



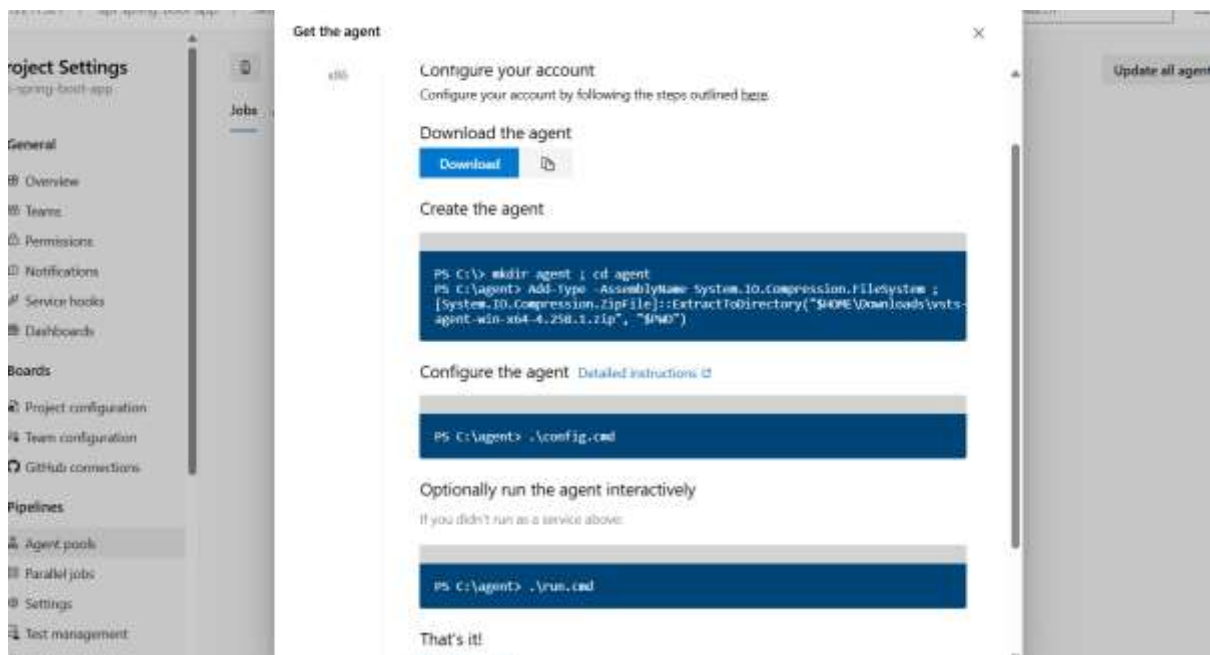
Service connection setup screen and Connection listed after creation.

I created a service connection to allow my pipelines to securely interact with Azure resources like App Services and ACR without manually providing credentials.

5. Create a Linux/Windows Self-Hosted Agent

Steps:

- Go to **Organization Settings > Agent Pools**.
- Create a new agent pool (if needed), then click on it and select **New agent**.
- Choose OS (Windows/Linux), download the agent.
- Configure and run using the given script (./config.sh or config.cmd), then run agent service.



Agent pool creation

- Similarly, add **Post-deployment approvers**.

Screenshot Suggestions:

- Pre/post-approver setup screen.

Explanation:

I added approval checks to ensure manual validation before and after deployments. This enforces governance and control over release processes.

7. CI/CD: Build & Push Docker Image to ACR & Deploy to AKS

Steps:

- Create Azure Container Registry (ACR) and Azure Kubernetes Service (AKS).
- Write a YAML pipeline:
 - Build Docker image
 - Push to ACR
 - Deploy using kubectl or Helm

```

13  stages:
14  - stage: BuildPush
15    jobs:
16    - job: DockerBuildPush
17      pool:
18        vmImage: ubuntu-latest
19      steps:
20      - task: Docker@2
21        inputs:
22          containerRegistry: 'scn4api' # ACR service connection
23          repository: '${appImageName}'
24          command: 'buildAndPush'
25          Dockerfile: 'backend/Dockerfile'
26          tags: |
27            $(appImageTag)
28
29  - stage: DeployACI
30    jobs:
31    - job: Deploy
32      pool:
33        vmImage: ubuntu-latest
34      steps:
35      - task: AzureCLI@2
36        inputs:
37          azureSubscription: 'arm4api' # Azure Resource Manager connection
38          scriptType: 'bash'

```

- Docker build and push logs.
- AKS deployment log or running pod.

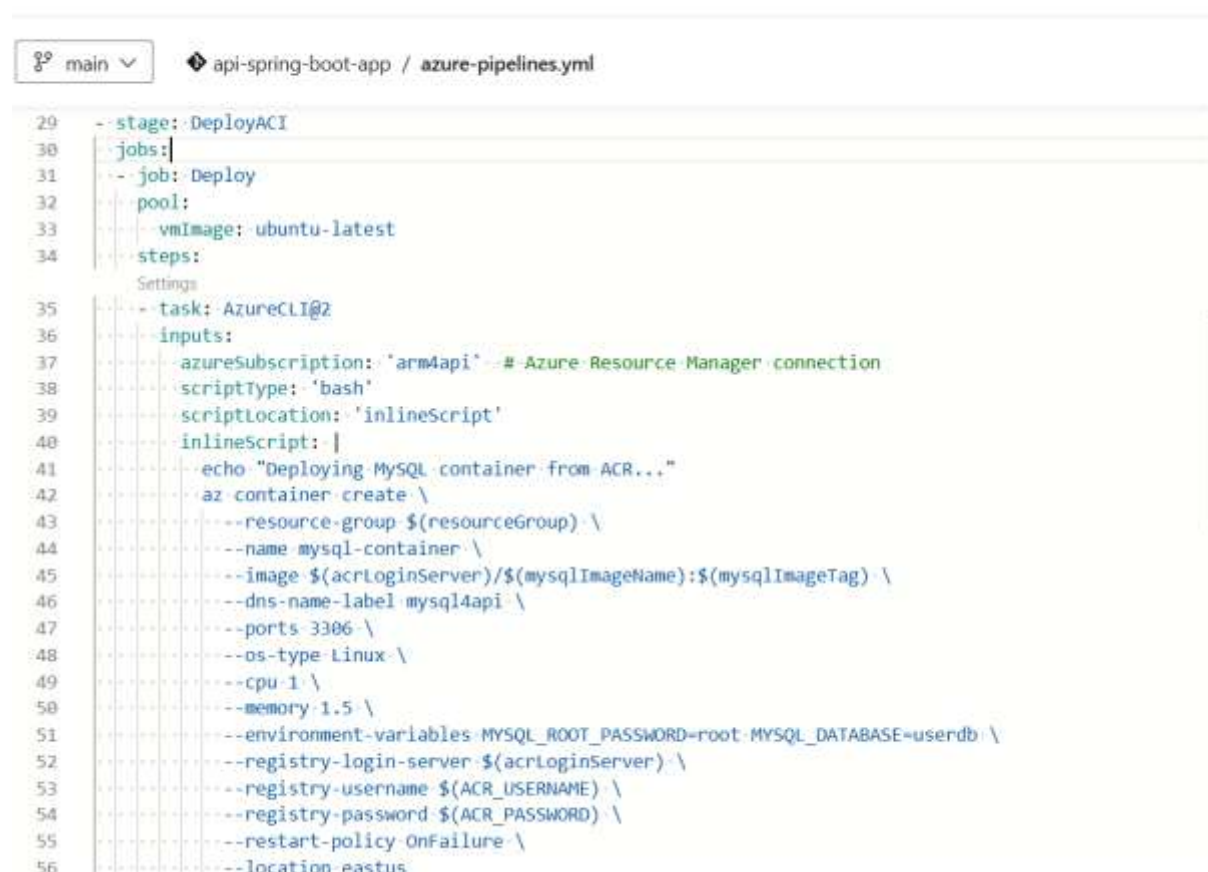
This pipeline automates the entire container lifecycle — from build to deployment — making AKS-based microservices deployment seamless and reliable.

8. CI/CD: Build & Push Docker Image to ACR & Deploy to ACI

Steps:

- Create ACR and ACI.
- Use YAML pipeline to:
 - Build and push Docker image to ACR.
 - Use Azure CLI task to deploy ACI.

← api-spring-boot-app



```
29 - stage: DeployACI
30   jobs:
31     - job: Deploy
32       pool:
33         vmImage: ubuntu-latest
34       steps:
35         - task: AzureCLI@2
36           inputs:
37             azureSubscription: 'arm4api' # Azure Resource Manager connection
38             scriptType: 'bash'
39             scriptLocation: 'inlineScript'
40             inlineScript: |
41               echo "Deploying MySQL container from ACR..."
42               az container create \
43                 --resource-group $(resourceGroup) \
44                 --name mysql-container \
45                 --image $(acrLoginServer)/$(mysqlImageName):$(mysqlImageTag) \
46                 --dns-name-label mysql4api \
47                 --ports 3306 \
48                 --os-type Linux \
49                 --cpu 1 \
50                 --memory 1.5 \
51                 --environment-variables MYSQL_ROOT_PASSWORD=root MYSQL_DATABASE=userdb \
52                 --registry-login-server $(acrLoginServer) \
53                 --registry-username $(ACR_USERNAME) \
54                 --registry-password $(ACR_PASSWORD) \
55                 --restart-policy OnFailure \
56                 --location eastus
```

ACR push log and ACI creation confirmation or running container screen.

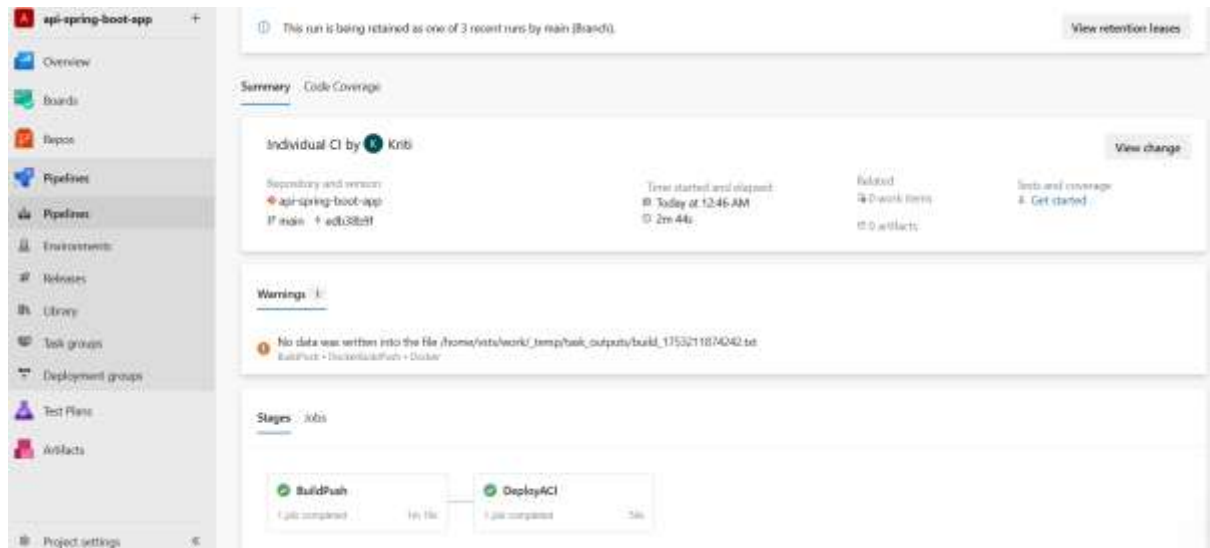
I deployed a lightweight container to Azure Container Instances for quick testing and cost-effective hosting using a fully automated pipeline.

9. CI/CD: Build .NET App and Deploy to Azure App Service

Steps:

- Create App Service and Azure DevOps pipeline.
- Use .NET Core tasks to build and publish the app.
- Deploy using Azure Web App task.

Build task success.



App Service deployment log or live URL.

I built and deployed a .NET web app using CI/CD, enabling fast and consistent deployments to Azure App Service with rollback capabilities.

10. CI/CD: Build React App and Deploy to Azure VM

Steps:

- Create and configure a Linux/Windows VM.
- Install NGINX or Node server.
- Build React app in pipeline.
- Use copy and SSH tasks to deploy files to the VM.

React build success and SSH file copy log and Webpage served from VM.

This pipeline demonstrates how to manually deploy static sites like React to a VM using SSH, giving full control over hosting and configuration.